

Richard Eidswick

11/11/2025

Foundations of Programming: Python - IT FDN 110 A

<https://github.com/Rasfer/IntroToProg-Python-Mod05>

Assignment 05

Introduction

For this week's assignment, I went through the steps of building a Python program that created an interactive registration menu that outputs information to a .json file. The key difference between the previous version was the introduction of working with dictionaries, the json file type, and error handling. After getting comfortable with these new topics, I used them to develop the program and test it in both PyCharm and the command prompt.

New Topics

Program

This assignment used an existing starter file that had most of the necessary variables, including the "MENU" global constant. Other key variables were the *student_data* and *students* that were list data types. I did remove the "csv_data" variable as it was no longer needed and change the "student_data" variable from a list to a dictionary. Another new component was adding in the "import" statement to bring in the json library (Figure 4).

```

9      # Import libraries
10     import json
11
12     # Define the Data Constants
13     MENU: str = '''
14     ---- Course Registration Program ----
15     Select from the following menu:
16     1. Register a Student for a Course.
17     2. Show current data.
18     3. Save data to a file.
19     4. Exit the program.
20     -----
21     '''
22     # Define the Data Constants
23     FILE_NAME: str = "Enrollments.json"
24
25     # Define the Data Variables and constants
26     student_first_name: str = '' # Holds the first name of a student entered by the user.
27     student_last_name: str = '' # Holds the last name of a student entered by the user.
28     course_name: str = '' # Holds the name of a course entered by the user.
29     student_data: dict = {} # one row of student data
30     students: list = [] # a table of student data
31     file = None # Holds a reference to an opened file.
32     menu_choice: str = '' # Hold the choice made by the user.
33

```

Figure 4: Import, constants, and variables.

The next part was the adding in error handling around opening the file. If the file did not exist, the program would print a statement and create the file (Figure 5).

```

34     try:
35         file = open(FILE_NAME, 'r')
36         students = json.load(file)
37         file.close()
38     except FileNotFoundError as e:
39         print("Text file must exist before running this script.")
40         print("Technical error message:")
41         print(e, e.__doc__, type(e), sep='\n')
42         file = open(FILE_NAME, 'w') # Creates the file if it didn't exist
43         file.write(json.dumps(students, indent=4)) # Creates the format for a list in the json file
44         file.close()
45     except Exception as e:
46         print("There was a non specific error")
47         print("Technical error message:")
48         print(e, e.__doc__, type(e), sep='\n')
49     finally:
50         if file.closed == False:
51             file.close()
52

```

Figure 5: Error handling around the file open.

Following the file opening section, the main *while* loop is run. The first selection choice goes through the user adding student data. Error handling is added around the name inputs to make sure the name is all alphabet characters. The program is now able to handle incorrect inputs and return the user back to the main menu (Figure 6).

```

61 # Input user data
62 if menu_choice == "1": # This will not work if it is an integer!
63     try:
64         # Input dat
65         student_first_name = input("Enter the student's first name: ")
66         if not student_first_name.isalpha():
67             raise ValueError("First name must be a letter")
68
69         student_last_name = input("Enter the student's last name: ")
70         if not student_last_name.isalpha():
71             raise ValueError("Last name must be a letter")
72
73         course_name = input("Enter course name: ")
74         # Creates the dictionary and loads it into the list
75         student_data = {"FirstName": student_first_name, "LastName": student_last_name, "CourseName": course_name}
76         students.append(student_data)
77         print(f'You have registered {student_data["FirstName"]} {student_data["LastName"]} for {student_data["CourseName"]}.')
78
79     # Exceptions
80     except ValueError as e:
81         print(e)
82         print("-- Technical error message --")
83         print(e.__doc__)
84         print(e.__str__())
85     except Exception as e:
86         print("There was a non-specific error")
87         print("Technical error message:")
88         print(e, e.__doc__, type(e), sep='\n')
89     continue

```

Figure 6: Error handling around the user's name.

Finally, the menu selection 3 is updated with json function and error handling while writing to the file. This should be unnecessary due to how the file is opened and read at the start of the program, but it will catch any type errors while writing the data to the file (Figure 7).

```

102     # Save the data to a file
103     elif menu_choice == "3":
104         try:
105             file = open(FILE_NAME, 'w')
106             json.dump(students, file, indent=4)
107             file.close()
108             continue
109         except TypeError as e:
110             print("Please check that the data is a valid JSON format\n")
111             print("-- Technical error message --")
112             print(e, e.__doc__, type(e), sep='\n')
113         except Exception as e:
114             print("There was a non specific error")
115             print("Built-In Python error info: ")
116             print(e, e.__doc__, type(e), sep='\n')
117         finally:
118             if file.closed == False:
119                 file.close()
120         print("Data Saved!")
121         continue

```

Figure 7: Error handling for writing to the file.

The last part was running the test. I ran the code through PyCharm with a check on the current data, adding a new student, checking the updated data, and loading it to the json file. The file started with a set of data from previous iterations (Figure 8).

```
---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----

What would you like to do: 2
-----

Student Vic Vu is enrolled in PY100.
Student Sue Jones is enrolled in PY100.
Student vic vu is enrolled in PY100.
Student Commander Prompt is enrolled in PY100.
Student Bjorn E is enrolled in GYM200.
Student vic vu is enrolled in py100.
-----
```

Figure 8: Starting the program by seeing the data in the file.

Next, the user accidentally enters test1 for the first name and is returned to the main menu (Figure 9).

```
---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----

What would you like to do: 1
Enter the student's first name: test1
First name must be a letter
-- Technical error message --
Inappropriate argument value (of correct type).
First name must be a letter

---- Course Registration Program ----
Select from the following menu:
```

Figure 9: Name with non-alphabetic characters.

The user then puts in the new student test tester in course PY9000 (Figure 10).

```
---- Course Registration Program ----  
Select from the following menu:  
  1. Register a Student for a Course.  
  2. Show current data.  
  3. Save data to a file.  
  4. Exit the program.  
-----  
  
What would you like to do: 1  
Enter the student's first name: test  
Enter the student's last name: tester  
Enter course name: PY9000  
You have registered test tester for PY9000.  
  
---- Course Registration Program ----
```

Figure 10: Entering in a new student.

Confirmation that the student was added to the data and is ready to load into the file (Figure 11).

```
What would you like to do: 2  
-----  
  
Student Vic Vu is enrolled in PY100.  
Student Sue Jones is enrolled in PY100.  
Student vic vu is enrolled in PY100.  
Student Commander Prompt is enrolled in PY100.  
Student Bjorn E is enrolled in GYM200.  
Student vic vu is enrolled in py100.  
Student test tester is enrolled in PY9000.  
-----  
  
---- Course Registration Program ----  
Select from the following menu:  
  1. Register a Student for a Course.  
  2. Show current data.  
  3. Save data to a file.  
  4. Exit the program.  
-----
```

Figure 11: Confirming student was added to the data.

Loading the data into the json file (Figure 12).

```
    },  
    {  
        "FirstName": "vic",  
        "LastName": "vu",  
        "CourseName": "py100"  
    },  
    {  
        "FirstName": "test",  
        "LastName": "tester",  
        "CourseName": "PY9000"  
    }  
]
```

Figure 12: Last entries in the "Enrollments.json" file.

Then, I ran the test again in command prompt (Figure 13).

```
C:\Users\reids\Documents\Career\School\Python 110\Module 5\_Module05\Assignment>Python Assignment05.py  
  
---- Course Registration Program ----  
Select from the following menu:  
1. Register a Student for a Course.  
2. Show current data.  
3. Save data to a file.  
4. Exit the program.  
-----  
  
What would you like to do: 2  
-----  
  
Student Vic Vu is enrolled in PY100.  
Student Sue Jones is enrolled in PY100.  
Student vic vu is enrolled in PY100.  
Student Commander Prompt is enrolled in PY100.  
Student Bjorn E is enrolled in GYM200.  
Student vic vu is enrolled in py100.  
Student test tester is enrolled in PY9000.  
-----
```

Figure 13: Running the program in command prompt.

Summary

In this assignment, I went through the process of creating a program that took input existing data from a .json file, added data from a user, and wrote to a file. I added error handling and the use of dictionaries to increase readability and efficiency of the code. It ran successfully in both PyCharm and command prompt.